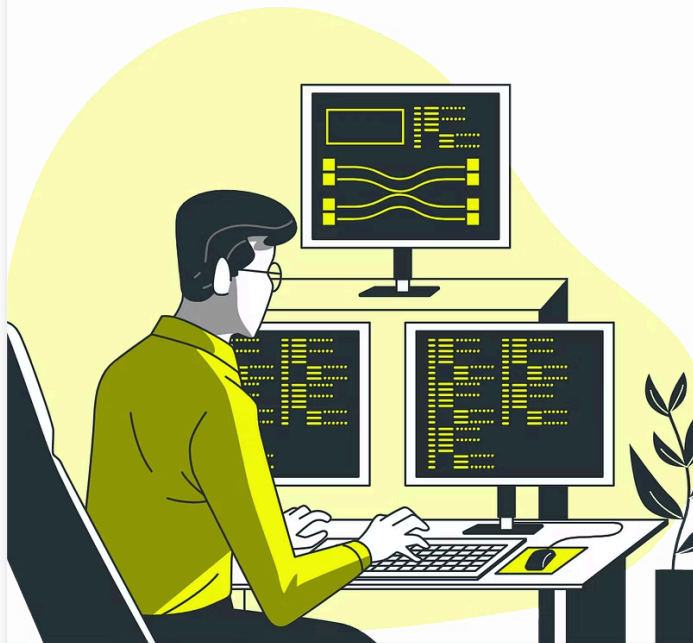




# Java

---

## Clase 10 | Introducción a Spring Boot

# ¡Les damos la bienvenida! 🚀

🎥 Vamos a comenzar a grabar la clase.

## Índice

---

**Clase 10**

**Clase 11**

## Clase 10 | Introducción a Spring Boot

- ¿Qué es Spring Boot y por qué utilizarlo?
- Creación de un proyecto Spring Boot desde cero.
- Estructura básica de un proyecto Spring Boot.
- Configuración inicial y dependencias necesarias.
- Primeros pasos con Spring Boot.

## Clase 11 | Desarrollo de la API REST con Spring Boot

- Concepto de API REST y métodos HTTP.
- Creación de controladores (@RestController).
- Manejo de rutas y endpoints básicos.
- Envío y recepción de datos en formato JSON.
- Implementación de operaciones CRUD en memoria.

# Objetivos de la Clase



### Comprender Spring y Spring Boot

Conocer el ecosistema Spring y las ventajas de Spring Boot.





## **Crear un proyecto Spring Boot**

Aprender a iniciar un proyecto desde cero con Spring Initializr.



## **Estructura y configuración**

Familiarizarse con la estructura básica y configuración inicial.



## **Implementar un endpoint REST**

Dar los primeros pasos creando un endpoint REST simple.

# Spring

---





# ¿Qué es Spring?

---

Spring es un framework de aplicación de código abierto para el desarrollo de aplicaciones Java empresariales. Su principal objetivo es simplificar y agilizar el desarrollo, promoviendo la modularidad y el desacoplamiento a través de la inyección de dependencias. Esto significa que las diferentes partes de una aplicación pueden ser desarrolladas e integradas de forma independiente, lo que facilita el mantenimiento, la prueba y la reutilización del código.



# Spring Framework

---



## Base del Ecosistema

Proporciona la infraestructura fundamental para aplicaciones Java robustas.



## Inyección de Dependencias

Facilita la gestión de componentes y sus dependencias.



## Programación AOP

Soporta programación orientada a aspectos para modularidad avanzada.

# Ecosistema Spring

---



## Spring MVC

Facilita la creación de aplicaciones web y controladores para endpoints HTTP.



## Spring Data

Simplifica el acceso a datos, integrando repositorios CRUD.



## Spring Security

Proporciona autenticación, autorización y otras características de seguridad.



## Otros Proyectos

Spring Integration, Spring Batch, Spring Cloud extienden las capacidades.



# Spring Boot

---

# ¿Qué es Spring Boot?

---

Spring Boot simplifica y acelera el desarrollo de aplicaciones Spring, eliminando la complejidad de la configuración. Se basa en convenciones sobre configuración, automatizando gran parte del proceso de configuración.

Su autoconfiguración analiza las dependencias y configura automáticamente las partes necesarias de la aplicación, ahorrando tiempo y esfuerzo.



# Beneficios de Spring Boot: Servidor Embebido y Startups Rápidos

---

## Servidor Integrado

No requiere despliegue en servidor externo. Se ejecuta con `java -jar`.

## Opciones de Servidor

Tomcat por defecto, pero soporta Jetty o Undertow.

## Desarrollo Ágil

Permite probar la API inmediatamente después de un commit.



# Creación de un Proyecto Spring Boot

---



## Spring Initializr

Visitar [start.spring.io](https://start.spring.io) para iniciar el proyecto.



## Configuración

Elegir versión, nombre, lenguaje y dependencias.



## Descarga

Obtener el proyecto generado.



## IDE

Abrir el proyecto en tu entorno de desarrollo preferido.

# Estructura Básica del Proyecto

---

Un proyecto Spring Boot básico usualmente incluye carpetas para la configuración (`src/main/resources`), código fuente (`src/main/java`), recursos de prueba (`src/test/java` y `src/test/resources`), y una carpeta para las dependencias (`pom.xml` en Maven o `build.gradle` en Gradle). Dentro de la carpeta `src/main/java` se encuentran los paquetes con el código Java de la aplicación.

## Directorios Principales

- `src/main/java`: Código fuente del backend
- `src/main/resources`: Archivos de configuración
- `src/test`: Pruebas unitarias e integración

## Archivos Clave

- `pom.xml` o `build.gradle`: Dependencias y configuración
- `application.properties`: Configuración de la aplicación
- Clase principal con `@SpringBootApplication`

# Dependencias Comunes

---



## **spring-boot-starter-web**

Para crear endpoints REST y aplicaciones web.



## **spring-boot-starter-data-jpa**

Para acceso a datos con JPA.



## **spring-boot-starter-security**

Para implementar seguridad en el backend.



## **spring-boot-starter-test**

Para pruebas unitarias e integración.

# Configuración Inicial



La configuración inicial es crucial para el correcto funcionamiento de una aplicación Spring Boot. Define aspectos fundamentales como la conexión a la base de datos, el puerto del servidor y otras propiedades esenciales que determinan el comportamiento de la aplicación. Vamos a ver dos maneras comunes de realizar esta configuración:

### **application.properties**

Archivo principal de configuración. Define propiedades como el puerto del servidor o conexión a base de datos.

### **application.yml**

Alternativa a .properties con formato YAML. Ofrece una estructura más legible para configuraciones complejas.

# Ejemplos de cada tipo de archivo:

## **application.properties**

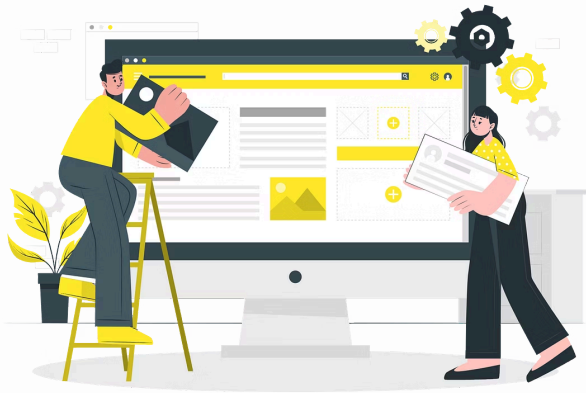
```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/mydatabase
```

## **application.yml**

```
server:
  port: 8080
spring:
  datasource:
    url:
      jdbc:mysql://localhost:3306/mydatabase
```

# Perfiles de Configuración

---



Los perfiles de configuración permiten adaptar la aplicación a diferentes entornos (desarrollo, pruebas, producción) mediante la definición de conjuntos de propiedades específicos para cada uno. Esto facilita la gestión de la configuración y garantiza que la aplicación funcione correctamente en cada entorno sin necesidad de modificaciones manuales.

## **Desarrollo**

Configuración para entorno local de desarrollo.

## **Pruebas**

Configuración para ejecutar pruebas automatizadas.

## **Producción**

Configuración optimizada para el entorno de producción.



# Primeros Pasos con Spring Boot

---

```
@SpringBootApplication
public class TechlabApplication {
    public static void main(String[] args) {

        SpringApplication.run(TechlabApplication.class,
            args);
    }
}
```

Esta clase inicia la aplicación Spring Boot. La anotación `@SpringBootApplication` configura automáticamente el proyecto.

# Creación de un Endpoint REST

---

```
@RestController
@RequestMapping("/productos")
public class ProductoController {
    @GetMapping
    public List listarProductos() {
        return Arrays.asList(new Producto("Café", 250.0, 100));
    }
}
```

Este controlador crea un endpoint GET en /productos que devuelve una lista de productos.



# Inyección de dependencias

---

# Inyección de Dependencias

---

La inyección de dependencias (DI) es un patrón de diseño que permite desacoplar las clases y hacer que sean más fáciles de probar. En lugar de que una clase cree sus propias dependencias (como instanciar objetos de otras clases), DI facilita que una clase externa (como el contenedor Spring) proporcione estas dependencias.

La inyección de dependencias se realiza utilizando el contenedor Spring, que es responsable de crear y gestionar las dependencias de las clases. El contenedor Spring utiliza la anotación `@Autowired` para inyectar las dependencias en las clases que las necesitan.



# Inyección de Dependencias

---

## @Autowired

Anotación para inyectar dependencias automáticamente. Spring Boot resuelve y proporciona la instancia necesaria.

## Ejemplo

```
@Service
public class ProductoService { ... }

@RestController
public class ProductoController {
    private final ProductoService
    productoService;

    @Autowired
    public
    ProductoController(ProductoService
    productoService) {
        this.productoService =
    productoService;
    }
}
```

# Materialles y Recursos Adicionales

---



## Documentación Oficial

Visita <https://spring.io/projects/spring-boot> para la documentación completa.



## Tutoriales en Video

Explora tutoriales en YouTube sobre Spring Boot y creación de APIs REST. <https://youtu.be/kFlvslQQZ9k?si=vHPRSt4l9iH-TecE>



## Comunidad Spring

Participa en foros y grupos de discusión de la comunidad Spring.

Haga clic aquí

# Preguntas para Reflexionar



## **Integración con Ecosistema**

¿Cómo se integra Spring Boot con el ecosistema Spring para agilizar el desarrollo?



## **Ventajas de Servidor Embebido**

¿Qué ventajas ofrece el arranque rápido y el servidor embebido?



## **Impacto en TechLab**

¿Cómo reduce la autoconfiguración la carga cognitiva del equipo en TechLab?

# **Ejercicios Prácticos**

---

# Situación inicial en TechLab.

---



Silvia, la Product Owner, quiere exponer la lógica del backend a un frontend o a clientes externos. Matías y Sabrina han oído hablar de Spring Boot y cómo acelera la creación de APIs REST. Buscan un framework maduro, con buena documentación, amplio soporte en la industria y extensible para necesidades futuras.

El equipo nota que con Spring Boot pueden arrancar un servidor en minutos, exponer endpoints, conectarse a una base de datos y añadir seguridad progresivamente. Esto reducirá el tiempo de configuración manual y les permitirá centrarse en la lógica del negocio, justo lo que TechLab necesita para escalar y ofrecer servicios estables

## Ejercicio Práctico

---

Optativo | No entregable

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

## Crea un proyecto Spring boot con Spring Initializr

- Selecciona Spring Web y Spring Data JPA.
- Descarga el proyecto, importalo en tu IDE.
- Agregá un endpoint GET /hello que devuelva un mensaje simple.

Opcional: Configurar un Puerto Diferente:

- En [application.properties](#), pon server.port=9090.
- Verificá que la aplicación corra en <http://localhost:9090>.



## Ejercicio Práctico

Optativo | No entregable

### Injectar Dependencias

- Agregar un Servicio e Injectarlo: